

# Homework 09: NumPY and Matplotlib

EEC 3150

For each Python exercise, write a separate Python script. Submit all scripts on Blackboard under the appropriate assignment. Name your script files with the following format:

HW<assignment number>\_Ex<exercise  
number>\_<FirstInitial><LastName>.py

Example: HW01\_Ex01\_GWest.py

For short answer or math exercises, either do your work on a sheet of paper and upload a picture of it to Blackboard or submit some kind of text file or Word document to Blackboard. You can place all such exercises in a single document as long as you clearly label each exercise. These files should follow a similar naming convention as the Python scripts.

### Exercise 1 (10 points, Python)

Create and print the following array WITHOUT directly coding any of the elements (only use numpy functions or operations):

```
[[ 1  3  5  7 ]
 [ 9 11 13 15 ]
 [17 19 21 23 ]
 [25 27 29 31 ]]
```

### Exercise 2 (10 points, Python)

1. Write a function `Boundary(n, value)` which will create an  $n \times n$  matrix in which the elements on the top, bottom, left, and right boundaries are equal to `value` while the inside elements are equal to 0.
2. Demonstrate your function by calling it and printing the output

Here is an example: `Boundary(4, 1)` should produce:

```
[[ 1  1  1  1 ]
 [ 1  0  0  1 ]
 [ 1  0  0  1 ]
 [ 1  1  1  1 ]]
```

### Exercise 3 (10 points, Python)

1. Write a function `Truncate(arr)` which takes an array `arr` of integers of any shape and returns a new array with the following properties:
  - Set all values where `arr` is even to 0
  - Set all values where `arr` is odd to 1
2. Demonstrate your function by generating an array of random integers on calling your function on it. Print the output.

Here is an example:

`Truncate(np.array([-1, 2, 11, -4]))` should produce:

```
[1 0 1 0]
```

### Exercise 4 (20 points, Python)

Plot a function of your choice on a domain of your choice.

1. Use `np.linspace` to create an array of  $x$  values (you choose the parameters)
2. Create an array of  $y$  values by evaluating some function of your choice on the array of  $x$  values
3. Plot attributes
  - Plot the  $x, y$  data in two ways on the same figure: 1) a green dashed line and 2) red dots
  - Set the `markersize` and `linewidth` to something other than the default
  - Set `xlim`, `ylim` to some reasonable values to display the entire plot

### Exercise 5 (20 points, Python)

The exercise will have you generate random data, plot it, and index it.

1. Generate random data
  - Create 1D  $x, y$  arrays of 1000 random numbers between -2 and 2
2. Plot the data
  - Plot the  $x, y$  arrays as blue dots
3. Index the data
  - Through whatever means you choose, create  $x_2, y_2$  arrays which only have the points where  $-1.5 < x < 1$  and  $-1 < y < 0.9$
4. Plot the new data
  - Plot the  $x_2, y_2$  arrays on the same plot as red x's with an increased `markersize` for readability

More problems on next page...

### Exercise 6 (20 points, Python)

1. Create a function to generate a 2D rotation matrix (2D array) (NOTE: sin and cos take radians). For reference, here is the matrix:

$$\begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

2. Create a rotation matrix  $R$  with  $\theta = 2\pi/n$  where  $n$  is an integer variable of your choice (it should be  $10 \leq n \leq 100$ )
3. Create the vector (1D array)  $x = [1 \ 0]$
4. Create an array of vectors (2D array) by iteratively applying the rotation matrix to the vector  $x$ . This should produce the points on the unit circle
5. Plot the points